Chương 14

Hồi quy logistic

14.1. Giới thiệu

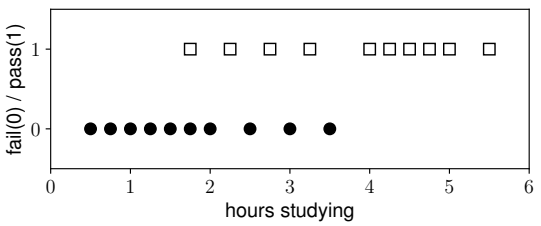
14.1.1. Nhắc lại hai mô hình tuyến tính

Hai mô hình tuyến tính đã thảo luận trong cuốn sách này, hồi quy tuyến tính và PLA, đều có thể viết chung dưới dạng $y = f(\mathbf{x}^T\mathbf{w})$ trong đó $f(s)$ là một hàm kích hoạt. Trong hồi quy tuyến tính $f(s) = s$, tích vô hướng $\mathbf{x}^T\mathbf{w}$ được trực tiếp sử dụng để dự đoán đầu ra y. Mô hình này phù hợp nếu ta cần dự đoán một đầu ra không bị chặn. PLA có đầu ra chỉ nhận một trong hai giá trị 1 hoặc -1 với hàm kích hoạt $f(s) = \text{sgn}(s)$ phù hợp với các bài toán phân loại nhị phân. Trong chương này, chúng ta sẽ thảo luận một mô hình tuyến tính với một hàm kích hoạt khác, thường được áp dụng cho các bài toán phân loại nhị phân. Trong mô hình này, đầu ra có thể được biểu diễn dưới dạng xác suất. Ví dụ, xác suất thi đỗ nếu biết thời gian ôn thi, xác suất ngày mai có mưa dựa trên những thông tin đo được trong ngày hôm nay,... Mô hình này có tên là *hồi quy logistic*. Mặc dù trong tên có chứa từ *hồi quy*, phương pháp này thường được sử dụng nhiều hơn cho các bài toán phân loại.

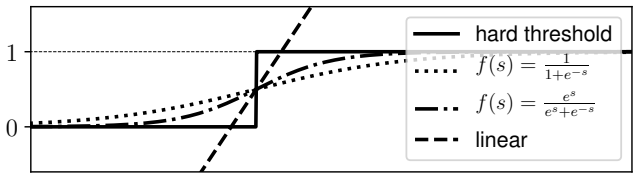
14.1.2. Một ví dụ nhỏ

Bảng 14.1: Thời gian ôn thi và kết quả thi của 20 sinh viên.

Số giờ	**Đậu?**	**Số giờ**	**Đậu?**	**Số giờ**	**Đậu?**	**Số giờ**	**Đậu?**
0.5	0	0.75	0	1	0	1.25	0
1.5	0	1.75	0	1.75	1	2	0
2.25	1	2.5	0	2.75	1	4	0
3.25	1	3.5	0	4	1	4.25	1
4.5	1	4.75	1	5	1	5.5	1



Hình 14.1. Ví dụ về kết quả thi dựa trên số giờ ôn tập. Trục hoành thể hiện thời gian ôn tập của mỗi sinh viên, trục tung gồm hai giá trị 0/fail (các điểm hình tròn) và 1/pass (các điểm hình vuông).



Hình 14.2. Một vài ví dụ về các hàm kích hoạt khác nhau.

Xét một ví dụ về quan hệ giữa thời gian ôn thi và kết quả của 20 sinh viên trong Bảng 14.1. Bài toán đặt ra là từ dữ liệu này hãy xây dựng mô hình đánh giá khả năng đỗ của một sinh viên dựa trên thời gian ôn tập. Dữ liệu trong Bảng 14.1 được mô tả trên Hình 14.1. Nhìn chung, thời gian học càng nhiều thì khả năng đỗ càng cao. Tuy nhiên, không có một ngưỡng thời gian học nào giúp phân biệt rạch ròi việc đỗ/trượt . Nói cách khác, dữ liệu của hai tập này là không tách biệt tuyến tính, và vì vậy PLA sẽ không hữu ích. Tuy nhiên, thay vì dự đoán chính xác hai giá trị đỗ/trượt, ta có thể dự đoán xác suất để một sinh viên thi đỗ dựa trên thời gian ôn thi.

14.1.3. Mô hình hồi quy logistic

Quan sát Hình 14.2 với các hàm kích hoạt $f(s)$ khác nhau.

- Đường nét đứt biểu diễn một hàm kích hoạt tuyến tính không phù hợp vì đầu ra không bị chặn. Có một cách đơn giản để đưa đầu ra về dạng bị chặn: nếu đầu ra nhỏ hơn không thì thay bằng không, nếu đầu ra lớn hơn một thì thay bằng một. Điểm phân chia, còn gọi là *ngưỡng*, được chọn là điểm có tung độ 0.5 trên đường thẳng này. Đây cũng không phải là một lựa chọn tốt. Giả sử có thêm một bạn sinh viên tiêu biểu ôn tập đến 20 giờ hoặc hơn thi đỗ. Lúc này ngưỡng tương ứng với mốc tung độ bằng 0.5 sẽ dịch nhiều về phía phải. Kéo theo đó, rất nhiều sinh viên thi đỗ được dự đoán là trượt. Rõ ràng đây là một mô hình không tốt. Nhắc lại rằng hồi quy tuyến tính rất nhạy cảm với nhiễu, ở đây là bạn sinh viên tiêu biểu đó.

- Đường nét liền tương tự với hàm kích hoạt của PLA[38]. Ngưỡng dự đoán đỗ/trượt tại vị trí hàm số đổi dấu còn được gọi là *ngưỡng cứng*.

[38] Đường này chỉ khác hàm kích hoạt của PLA ở chỗ hai nhãn là 0 và 1 thay vì -1 và 1.

- Các đường nét chấm và chấm gạch phù hợp với bài toán đang xét hơn. Chúng có một vài tính chất quan trọng:
  - Là các hàm số liên tục nhận giá trị thực, bị chặn trong khoảng $(0, 1)$.
  - Nếu coi điểm có tung độ bằng 0.5 là ngưỡng, các điểm càng xa ngưỡng về bên trái có giá trị càng gần không, các điểm càng xa ngưỡng về bên phải có giá trị càng gần một. Điều này phù hợp với nhận xét rằng học càng nhiều thì xác suất đỗ càng cao và ngược lại.
  - Hai hàm này có đạo hàm mọi nơi, điều này có thể có ích trong tối ưu.

Hàm sigmoid và tanh

Trong các hàm số có ba tính chất nói trên, hàm *sigmoid*:

$$f(s) = \frac{1}{1 + e^{-s}} \triangleq \sigma(s) \tag{14.1}$$

được sử dụng nhiều nhất, vì nó bị chặn trong khoảng $(0, 1)$ và:

$$\lim_{s \to -\infty} \sigma(s) = 0; \quad \lim_{s \to +\infty} \sigma(s) = 1. \tag{14.2}$$

Thú vị hơn:

$$\sigma'(s) = \frac{e^{-s}}{(1 + e^{-s})^2} = \frac{1}{1 + e^{-s}} \frac{e^{-s}}{1 + e^{-s}} = \sigma(s)(1 - \sigma(s)) \tag{14.3}$$

Với đạo hàm đơn giản, hàm sigmoid được sử dụng rộng rãi trong mạng neuron. Chúng ta sẽ sớm thấy hàm sigmoid được khám phá ra như thế nào.

Ngoài ra, hàm *tanh* cũng hay được sử dụng:

$$\tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}} = 2\sigma(2s) - 1. \tag{14.4}$$

Hàm số này nhận giá trị trong khoảng $(-1, 1)$.

Hàm sigmoid có thể được thực hiện trên Python như sau:

```
def sigmoid(S):
    """
    S: an numpy array
    return sigmoid function of each element of S
    """
    return 1/(1 + np.exp(-S))
```

14.2. Hàm mất mát và phương pháp tối ưu

14.2.1. Xây dựng hàm mất mát

Với các mô hình có hàm kích hoạt $f(s) \in (0, 1)$, ta có thể giả sử rằng xác suất để một điểm dữ liệu $\mathbf{x}_i$ có nhãn thứ nhất là $f(\mathbf{x}_i^T\mathbf{w})$ và nhãn còn lại là $1 - f(\mathbf{x}_i^T\mathbf{w})$:

$$p(y_i = 1|\mathbf{x}_i; \mathbf{w}) = f(\mathbf{x}_i^T\mathbf{w}) \tag{14.5}$$

$$p(y_i = 0|\mathbf{x}_i; \mathbf{w}) = 1 - f(\mathbf{x}_i^T\mathbf{w}) \tag{14.6}$$

trong đó $p(y_i = 1|\mathbf{x}_i; \mathbf{w})$ được hiểu là xác suất xảy ra sự kiện nhãn $y_i = 1$ khi biết tham số mô hình $\mathbf{w}$ và dữ liệu đầu vào $\mathbf{x}_i$. Mục đích là tìm các hệ số $\mathbf{w}$ sao cho $f(\mathbf{x}_i^T\mathbf{w}) \approx y_i$ với mọi điểm trong tập huấn luyện.

Ký hiệu $a_i = f(\mathbf{x}_i^T\mathbf{w})$, hai biểu thức (14.5) và (14.6) có thể được viết gọn lại:

$$p(y_i|\mathbf{x}_i; \mathbf{w}) = a_i^{y_i}(1 - a_i)^{1-y_i} \tag{14.7}$$

Biểu thức này tương đương với hai biểu thức (14.5) và (14.6) vì khi $y_i = 1$, thừa số thứ hai của vế phải sẽ bằng một, khi $y_i = 0$, thừa số thứ nhất sẽ bằng một. Để mô hình tạo ra dự đoán khớp với dữ liệu đã cho nhất, ta cần tìm $\mathbf{w}$ để xác xuất này đạt giá trị cao nhất.

Xét toàn bộ tập huấn luyện với ma trận dữ liệu $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N] \in \mathbb{R}^{d\times N}$ và vector nhãn tương ứng $\mathbf{y} = [y_1, y_2, \ldots, y_N]$. Ta cần giải bài toán tối ưu

$$\mathbf{w} = \arg\max_{\mathbf{w}} p(\mathbf{y}|\mathbf{X}; \mathbf{w}) \tag{14.8}$$

Đây chính là một bài toán MLE với tham số mô hình $\mathbf{w}$ cần được ước lượng. Ta có thể giải quyết bài toán này bằng cách giả sử các điểm dữ liệu độc lập nếu biết tham số mô hình. Đây cũng là giả sử thường được dùng khi giải các bài toán liên quan tới MLE:

$$p(\mathbf{y}|\mathbf{X}; \mathbf{w}) = \prod_{i=1}^{N} p(y_i|\mathbf{x}_i; \mathbf{w}) = \prod_{i=1}^{N} a_i^{y_i}(1 - a_i)^{1-y_i} \tag{14.9}$$

Lấy logarit tự nhiên, đổi dấu rồi lấy trung bình, ta thu được hàm số

$$J(\mathbf{w}) = -\frac{1}{N}\log p(\mathbf{y}|\mathbf{X}; \mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N}(y_i \log a_i + (1 - y_i)\log(1 - a_i)) \tag{14.10}$$

với chú ý rằng a_i là một hàm số của $\mathbf{w}$ và $\mathbf{x}_i$. Hàm số này là hàm mất mát của hồi quy logistic. Vì đã đổi dấu sau khi lấy logarit, ta cần tìm $\mathbf{w}$ để $J(\mathbf{w})$ đạt giá trị nhỏ nhất.

14.2.2. Tối ưu hàm mất mát

Bài toán tối ưu hàm mất mát của hồi quy logistic có thể được giải quyết bằng SGD. Tại mỗi vòng lặp, $\mathbf{w}$ được cập nhật dựa trên một điểm dữ liệu ngẫu nhiên. Hàm mất mát của hồi quy logistic với chỉ một điểm dữ liệu $(\mathbf{x}_i, y_i)$ và gradient của nó lần lượt là

$$J(\mathbf{w}; \mathbf{x}_i, y_i) = -(y_i \log a_i + (1 - y_i) \log(1 - a_i)) \tag{14.11}$$

$$\nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{x}_i, y_i) = -(\frac{y_i}{a_i} - \frac{1 - y_i}{1 - a_i})(\nabla_{\mathbf{w}} a_i) = \frac{a_i - y_i}{a_i(1 - a_i)}(\nabla_{\mathbf{w}} a_i) \tag{14.12}$$

ở đây ta đã sử dụng quy tắc chuỗi để tính gradient với $a_i = f(\mathbf{x}_i^T \mathbf{w})$. Để cho biểu thức này đơn giản, ta sẽ tìm hàm $a_i = f(\mathbf{x}_i^T \mathbf{w})$ sao cho mẫu số bị triệt tiêu.

Đặt $z = \mathbf{x}_i^T \mathbf{w}$, ta có

$$\nabla_{\mathbf{w}} a_i = \frac{\partial a_i}{\partial z_i}(\nabla_{\mathbf{w}} z_i) = \frac{\partial a_i}{\partial z_i}\mathbf{x}_i \tag{14.13}$$

Tạm thời bỏ qua các chỉ số i, ta đi tìm hàm số $a = f(z)$ sao cho

$$\frac{\partial a}{\partial z} = a(1 - a) \tag{14.14}$$

Nếu điều này xảy ra, mẫu số trong biểu thức (14.12) sẽ bị triệt tiêu. Phương trình vi phân này không quá phức tạp. Thật vậy, (14.14) tương đương với

$$\begin{aligned}
& \frac{\partial a}{a(1-a)} = \partial z \\
\Leftrightarrow \quad & \left(\frac{1}{a} + \frac{1}{1-a}\right)\partial a = \partial z \\
\Leftrightarrow \quad & \log a - \log(1-a) = z + C \\
\Leftrightarrow \quad & \log \frac{a}{1-a} = z + C \\
\Leftrightarrow \quad & \frac{a}{1-a} = e^{z+C} \\
\Leftrightarrow \quad & a = e^{z+C}(1-a) \\
\Leftrightarrow \quad & a = \frac{e^{z+C}}{1+e^{z+C}} = \frac{1}{1+e^{-z-C}} = \sigma(z+C)
\end{aligned}$$

với C là một hằng số. Chọn $C = 0$, ta được $a = f(\mathbf{x}^T \mathbf{w}) = \sigma(z)$. Đây chính là hàm sigmoid. Hồi quy logistic với hàm kích hoạt là hàm sigmoid được sử dụng phổ biến nhất. Mô hình này còn có tên là *hồi quy logistic sigmoid*. Khi nói hồi quy logistic, ta ngầm hiểu rằng đó chính là hồi quy logistic sigmoid.

Thay (14.13) và (14.14) vào (14.12) ta thu được

$$\nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{x}_i, y_i) = (a_i - y_i)\mathbf{x}_i = (\sigma(\mathbf{x}_i^T \mathbf{w}) - y_i)\mathbf{x}_i. \tag{14.15}$$

Từ đó, công thức cập nhật nghiệm cho hồi quy logistic sử dụng SGD là

$$\mathbf{w} \leftarrow \mathbf{w} - \eta(a_i - y_i)\mathbf{x}_i = \mathbf{w} - \eta(\sigma(\mathbf{x}_i^T\mathbf{w}) - y_i)\mathbf{x}_i \tag{14.16}$$

với η là tốc độ học.

14.2.3. Hồi quy logistic với suy giảm trọng số

Một trong các kỹ thuật phổ biến giúp tránh overfitting cho các mạng neuron là sử dụng *suy giảm trọng số* (*weight decay*). Đây là một kỹ thuật kiểm soát, trong đó một đại lượng tỉ lệ với bình phương chuẩn ℓ_2 của vector trọng số $\mathbf{w}$ được cộng vào hàm mất mát để kiểm soát độ lớn của các hệ số. Hàm mất mát trở thành

$$\bar{J}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\left(-y_i \log a_i - (1-y_i)\log(1-a_i) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2\right). \tag{14.17}$$

Công thức cập nhật $\mathbf{w}$ bằng SGD trong hồi quy logistic với suy giảm trọng số là:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\left((\sigma(\mathbf{x}_i^T\mathbf{w}) - y_i)\mathbf{x}_i + \lambda\mathbf{w}\right) \tag{14.18}$$

14.3. Triển khai thuật toán trên Python

Hàm ước lượng xác suất đầu ra cho mỗi điểm dữ liệu và hàm tính giá trị hàm mất mát với weight decay có thể được thực hiện như sau trong Python.

```
def prob(w, X):
    """
    X: a 2d numpy array of shape (N, d). N datatpoint, each with size d
    w: a 1d numpy array of shape (d)
    """
    return sigmoid(X.dot(w))

def loss(w, X, y, lam):
    """
    X, w as in prob
    y: a 1d numpy array of shape (N). Each elem = 0 or 1
    """
    a = prob(w, X)
    loss_0 = -np.mean(y*np.log(a) + (1-y)*np.log(1-a))
    weight_decay = 0.5*lam/X.shape[0]*np.sum(w*w)
    return loss_0 + weight_decay
```

Từ công thức (14.18), ta có thể thực hiện thuật toán tìm $\mathbf{w}$ cho hồi quy logistic như sau:

```
def logistic_regression(w_init, X, y, lam, lr = 0.1, nepoches = 2000):
    # lam: regulariza parameter, lr: learning rate, nepoches: # epoches
    N, d = X.shape[0], X.shape[1]
    w = w_old = w_init
    # store history of loss in loss_hist
    loss_hist = [loss(w_init, X, y, lam)]
    ep = 0
    while ep < nepoches:
        ep += 1
        mix_ids = np.random.permutation(N) # stochastic
        for i in mix_ids:
            xi = X[i]
            yi = y[i]
            ai = sigmoid(xi.dot(w))
            # update
            w = w - lr*((ai - yi)*xi + lam*w)
            loss_hist.append(loss(w, X, y, lam))
        if np.linalg.norm(w - w_old)/d < 1e-6:
            break
        w_old = w
    return w, loss_hist
```

14.3.1. Hồi quy logistic cho ví dụ đầu chương

Áp dụng vào bài toán dự đoán đỗ/trượt ở đầu chương:

```
np.random.seed(2)
X = np.array([[0.50, 0.75, 1.00, 1.25, 1.50, 1.75, 1.75, 2.00, 2.25, 2.50,
2.75, 3.00, 3.25, 3.50, 4.00, 4.25, 4.50, 4.75, 5.00, 5.50]]).T
y = np.array([0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1])

# bias trick
Xbar = np.concatenate((X, np.ones((X.shape[0], 1))), axis = 1)
w_init = np.random.randn(Xbar.shape[1])
lam = 0.0001
w, loss_hist = logistic_regression(w_init, Xbar, y, lam, lr = 0.05, nepoches
     = 500)
print('Solution of Logistic Regression:', w)
print('Final loss:', loss(w, Xbar, y, lam))
```

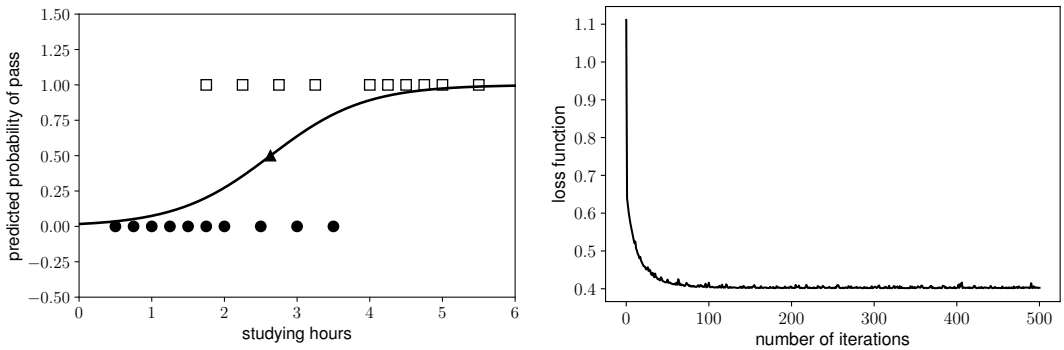
Kết quả:

```
Solution of Logistic Regression: [ 1.54337021 -4.06486702]
Final loss: 0.402446724975
```

Từ đây ta có thể rút ra xác suất thi đỗ dựa trên công thức:

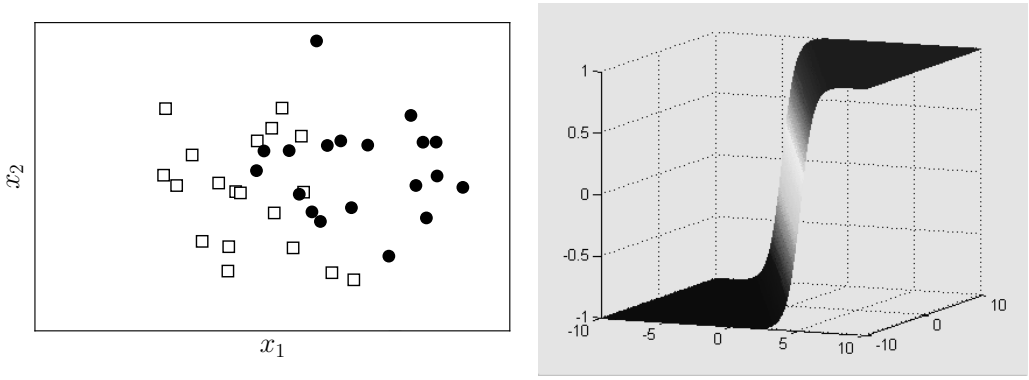
$$\texttt{probability_of_pass} \approx \texttt{sigmoid(1.54 * hours_of_studying - 4.06)}$$

Biểu thức này cũng chỉ ra rằng xác suất thi đỗ tăng khi thời gian ôn tập tăng, do sigmoid là một hàm đồng biến. Nghiệm của mô hình hồi quy logistic và giá trị hàm mất mát qua mỗi epoch được mô tả trên Hình 14.3.



(a) **(b)**

Hình 14.3. Nghiệm của hồi quy logistic cho bài toán dự đoán kết quả thi dựa trên thời gian học. (a) Đường nét liền thể hiện xác suất thi đỗ dựa trên thời gian học. Điểm tam giác thể hiện ngưỡng ra quyết định đỗ/trượt. Điểm này có thể thay đổi tuỳ vào bài toán. (b) Giá trị của hàm mất mát qua các vòng lặp. Hàm mất mát giảm nhanh và hội tụ sớm.



(a) Dữ liệu cho bài toán phân loại trong không gian hai chiều.

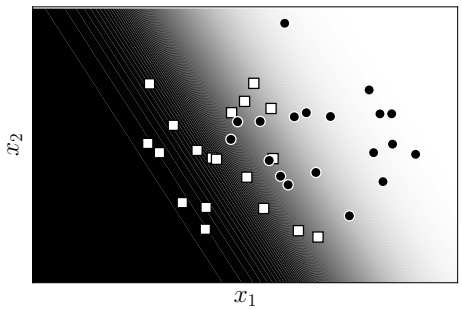
(b) Đồ thị hàm sigmoid trong không gian hai chiều (xem ảnh màu trong Hình B.6).

Hình 14.4. Ví dụ về dữ liệu và hàm sigmoid trong không gian hai chiều.

14.3.2. Ví dụ với dữ liệu hai chiều

Giả sử có hai tập dữ liệu vuông và tròn phân bố trên mặt phẳng như trong Hình 14.4a. Với dữ liệu đầu vào nằm trong không gian hai chiều, hàm sigmoid có dạng thác nước như trong Hình 14.4b.

Kết quả dự đoán xác suất đầu ra khi áp dụng mô hình hồi quy logistic được minh họa như Hình 14.5 với độ sáng của nền thể hiện xác suất điểm đó có nhãn tròn. Màu đen đậm thể hiện giá trị gần bằng không, màu trắng thể hiện giá trị rất gần bằng một.



Hình 14.5. Ví dụ về hồi quy logistic với dữ liệu hai chiều. Vùng màu càng đen thể hiện xác suất thuộc nhãn hình vuông càng cao. Vùng màu càng trắng thể hiện xác suất thuộc nhãn hình tròn càng cao. Vùng biên giữa hai nhãn (khu vực màu xám) thể hiện các điểm thuộc vào mỗi nhãn với xác suất thấp hơn.

Nếu phải lựa chọn một ranh giới thay vì xác suất, ta thấy các đường thẳng nằm trong khu vực màu xám là các lựa chọn hợp lý. Ta sẽ chứng minh ở phần sau rằng tập hợp các điểm có cùng xác suất đầu ra tạo thành một siêu phẳng.

Mã nguồn cho chương này có thể được tìm thấy tại https://goo.gl/9e7sPF.

Cách sử dụng hồi quy logistic trong thư viện scikit-learn có thể được tìm thấy tại https://goo.gl/BJLJNx.

14.4. Tính chất của hồi quy logistic

a. *Hồi quy logistic thực ra là một thuật toán phân loại.*

Mặc dù trong tên có từ *hồi quy*, hồi quy logistic được sử dụng nhiều trong các bài toán phân loại. Sau khi tìm được mô hình, việc xác định nhãn y cho một điểm dữ liệu $\mathbf{x}$ được xác định bằng việc so sánh hai giá trị:

$$P(y = 1|\mathbf{x}; \mathbf{w}); \quad P(y = 0|\mathbf{x}; \mathbf{w}) \tag{14.19}$$

Nếu giá trị thứ nhất lớn hơn, ta kết luận điểm dữ liệu có nhãn một và ngược lại. Vì tổng hai giá trị này luôn bằng một nên ta chỉ cần xác định $P(y = 1|\mathbf{x}; \mathbf{w})$ có lớn hơn 0.5 hay không.

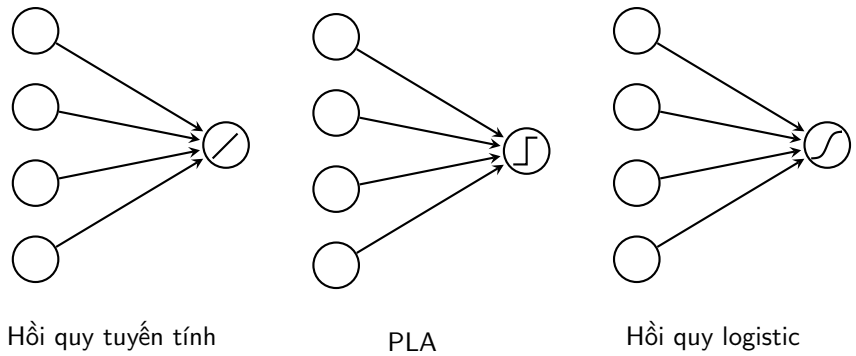
b. *Đường ranh giới tạo bởi hồi quy logistic là một siêu phẳng.*

Thật vậy, giả sử những điểm có xác suất đầu ra lớn hơn 0.5 được gán nhãn một. Tập hợp các điểm này là nghiệm của bất phương trình:

$$P(y = 1|\mathbf{x}; \mathbf{w}) > 0.5 \Leftrightarrow \frac{1}{1 + e^{-\mathbf{x}^T\mathbf{w}}} > 0.5 \Leftrightarrow e^{-\mathbf{x}^T\mathbf{w}} < 1 \Leftrightarrow \mathbf{x}^T\mathbf{w} > 0$$

Nói cách khác, tập hợp các điểm được gán nhãn một tạo thành một nửa không gian $\mathbf{x}^T\mathbf{w} > 0$, tập hợp các điểm được gán nhãn không tạo thành nửa không gian còn lại. Ranh giới giữa hai nhãn là siêu phẳng $\mathbf{x}^T\mathbf{w} = 0$.

Vì vậy, hồi quy logistic được coi như một bộ phân loại tuyến tính.



Hình 14.6. Biểu diễn hồi quy tuyến tính, PLA, và hồi quy logistic dưới dạng neural network.

c. *Hồi quy logistic không yêu cầu giả thiết tách biệt tuyến tính.*

Một điểm cộng của hồi quy logistic so với PLA là nó không cần giả thiết dữ liệu hai tập hợp là tách biệt tuyến tính. Tuy nhiên, ranh giới tìm được vẫn có dạng tuyến tính. Vì vậy, mô hình này chỉ phù hợp với loại dữ liệu mà hai tập *gần* tách biệt tuyến tính.

d. *Ngưỡng ra quyết định có thể thay đổi.*

Hàm dự đoán đầu ra của các điểm dữ liệu mới có thể được viết như sau:

```
def predict(w, X, threshold = 0.5):
"""
predict output for each row of X
X: a numpy array of shape (N, d), threshold: 0 < threshold < 1
return a 1d numpy array, each element is 0 or 1
"""
res = np.zeros(X.shape[0])
res[np.where(prob(w, X) > threshold)[0]] = 1
return res
```

Trong các ví dụ đã nêu, ngưỡng ra quyết định đều được lấy tại 0.5. Trong nhiều trường hợp, ngưỡng này có thể được thay đổi. Ví dụ, việc xác định các giao dịch là lừa đảo của một công ty tín dụng là rất quan trọng. Việc phân loại nhầm một giao dịch lừa đảo thành một giao dịch thông thường gây ra hậu quả nghiêm trọng hơn chiều ngược lại. Trong bài toán đó, ngưỡng phân loại có thể giảm xuống còn 0.3. Nghĩa là các giao dịch được dự đoán là lừa đảo với xác suất lớn hơn 0.3 sẽ được gán nhãn lừa đảo và cần được xử lý bằng các biện pháp khác.

e. Khi biểu diễn dưới dạng các mạng neuron, hồi quy tuyến tính, PLA và hồi quy logistic có thể được biểu diễn như trong Hình 14.6. Sự khác nhau chỉ nằm ở lựa chọn hàm kích hoạt.

14.5. Bài toán phân biệt hai chữ số viết tay

Xét bài toán phân biệt hai chữ số không và một trong bộ cơ sở dữ liệu MNIST. Trong mục này, class `LogisticRegression` trong thư viện scikit-learn sẽ được sử dụng. Trước tiên, ta cần khai báo các thư viện và tải về bộ cơ sở dữ liệu MNIST:

```
import numpy as np
from sklearn.datasets import fetch_mldata
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
mnist = fetch_mldata('MNIST original', data_home='../../data/')
N, d = mnist.data.shape
print('Total {:d} samples, each has {:d} pixels.'.format(N, d))
```

Kết quả:

```
Total 70000 samples, each has 784 pixels.
```

Có tổng cộng 70000 điểm dữ liệu trong tập dữ liệu MNIST, mỗi điểm là một mảng 784 phần tử tương ứng với 784 pixel. Mỗi chữ số từ không đến chín chiếm khoảng mười phần trăm. Chúng ta sẽ lấy ra tất cả các điểm ứng với chữ số không và một, sau đó chọn ngẫu nhiên 2000 điểm làm tập kiểm tra, phần còn lại đóng vai trò tập huấn luyện.

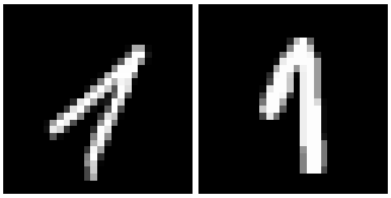
```
X_all = mnist.data
y_all = mnist.target
X0 = X_all[np.where(y_all == 0)[0]] # all digit 0
X1 = X_all[np.where(y_all == 1)[0]] # all digit 1
y0 = np.zeros(X0.shape[0]) # class 0 label
y1 = np.ones(X1.shape[0])  # class 1 label
X = np.concatenate((X0, X1), axis = 0) # all digits 0 and 1
y = np.concatenate((y0, y1)) # all labels
# split train and test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=2000)
```

Tiếp theo, ta xây dựng mô hình hồi quy logistic trên tập huấn luyện và dự đoán nhãn của các điểm trong tập kiểm tra. Kết quả này được so sánh với nhãn thực sự của mỗi điểm dữ liệu để tính độ chính xác của bộ phân loại:

```
model = LogisticRegression(C = 1e5) # C is inverse of lam
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy %.2f %%" % (100*accuracy_score(y_test, y_pred.tolist())))
```

Kết quả:

```
Accuracy 99.90 %
```



Hình 14.7. Các chữ số bị phân loại lỗi trong bài toán phân loại nhị phân với hai chữ số không và một.

Như vậy, gần 100% các ảnh được phân loại chính xác. Điều này dễ hiểu vì hai chữ số không và một khác nhau rất nhiều.

Tiếp theo, ta cùng đi tìm những ảnh bị phân loại sai và hiển thị chúng:

```
mis = np.where((y_pred - y_test) !=0)[0]
Xmis = X_test[mis, :]
from display_network import *
filename = 'mnist_mis.pdf'
with PdfPages(filename) as pdf:
plt.axis('off')
A = display_network(Xmis.T, 1, Xmis.shape[0])
f2 = plt.imshow(A, interpolation='nearest' )
plt.gray()
pdf.savefig(bbox_inches='tight')
plt.show()
```

Chỉ có hai chữ số bị phân loại lỗi được cho trên Hình 14.7. Trong đó, chữ số không bị phân loại lỗi là dễ hiểu vì nó trông rất giống chữ số một.

Bạn đọc có thể xem thêm ví dụ về bài toán xác định giới tính dựa trên ảnh khuôn mặt tại https://goo.gl/9V8wdD.

14.6. Bài toán phân loại đa lớp

Hồi quy logistic được áp dụng cho các bài toán phân loại nhị phân. Các bài toán phân loại thực tế có thể có nhiều hơn hai nhãn dữ liệu, được gọi là bài toán *phân loại đa lớp* (*multi-class classification*). Hồi quy logistic cũng có thể được áp dụng vào các bài toán này bằng một vài kỹ thuật.

Có ít nhất bốn cách áp dụng các bộ phân loại nhị phân vào bài toán phân loại đa lớp.

14.6.1. one-vs-one

Ta có thể xây dựng nhiều bộ phân loại nhị phân cho từng cặp hai nhãn dữ liệu. Bộ thứ nhất phân biệt nhãn thứ nhất và nhãn thứ hai, bộ thứ hai phân biệt nhãn thứ nhất và nhãn thứ ba,... Có tổng cộng $P = \frac{C(C-1)}{2}$ bộ phân loại nhị phân cần xây dựng với C là số lượng nhãn. Cách thực hiện này được gọi là *one-vs-one*.

Với một điểm dữ liệu kiểm tra, ta dùng tất cả P bộ phân loại để dự đoán nhãn của nó. Kết quả cuối cùng có thể được xác định bằng cách xem điểm dữ liệu đó được gán nhãn nào nhiều nhất. Ngoài ra, nếu mỗi bộ phân loại có thể đưa ra xác suất giống hồi quy logistic, ta có thể tính tổng các xác suất mà điểm dữ liệu đó rơi vào mỗi nhãn. Chú ý rằng tổng các xác suất là P thay vì một bởi có P bộ phân loại khác nhau.

Cách làm này không lợi về tính toán vì số bộ phân loại phải huấn luyện tăng nhanh khi số nhãn tăng lên. Hơn nữa, điều không hợp lý xảy ra nếu một chữ số có nhãn bằng một được đưa vào bộ phân loại giữa hai nhãn chữ số năm và sáu.

14.6.2. Phân loại phân tầng

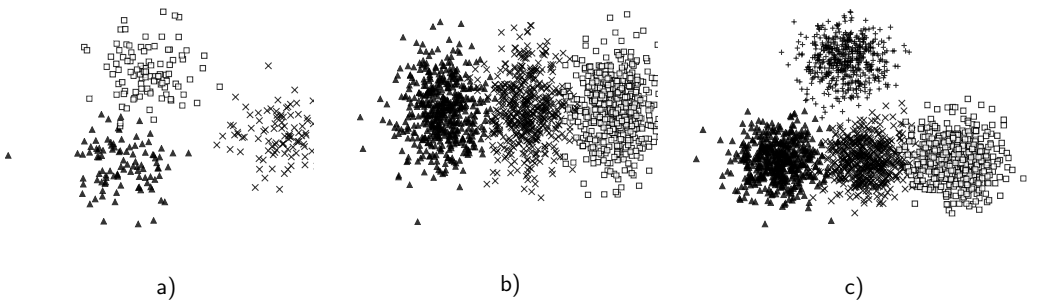
One-vs-one yêu cầu xây dựng $\frac{C(C-1)}{2}$ bộ phân loại khác nhau. Để giảm số bộ phân loại cần xây dựng, ta có thể dùng phương pháp *phân tầng*. Ý tưởng của phương pháp này có thể được thấy qua ví dụ sau.

Xét bài toán phân loại bốn chữ số {4, 5, 6, 7} trong MNIST. Vì chữ số 4 và 7 khá giống nhau, chữ số 5 và 6 khá giống nhau nên trước tiên ta xây dựng bộ phân loại giữa {4, 7} và {5, 6}. Sau đó xây dựng thêm hai bộ phân loại để xác định từng chữ số trong mỗi nhóm. Tổng cộng, ta cần ba bộ phân loại nhị phân so với sáu bộ như khi sử dụng one-vs-one.

Có nhiều cách chia nhỏ tập dữ liệu ban đầu ra các cặp tập con. Cách phân tầng có ưu điểm là giảm số bộ phân loại nhị phân cần xây dựng. Tuy nhiên, cách làm này có một hạn chế lớn: nếu chỉ một bộ phân loại cho kết quả sai thì kết quả cuối cùng chắc chắn sẽ sai. Ví dụ, nếu một ảnh chứa chữ số 5 bị phân loại lỗi bởi bộ phân loại đầu tiên thì cuối cùng nó sẽ bị nhận nhầm thành 4 hoặc 7.

14.6.3. Mã hoá nhị phân

Có một cách tiếp tục giảm số bộ phân loại là *mã hoá nhị phân*. Trong phương pháp này, mỗi nhãn được mã hoá bởi một số nhị phân. Ví dụ, nếu có bốn nhãn thì chúng được mã hoá bởi 00, 01, 10, và 11. Số bộ phân loại nhị phân cần xây dựng chỉ là $m = \lceil \log_2(C) \rceil$ trong đó C là số nhãn, $\lceil a \rceil$ là số nguyên nhỏ nhất không nhỏ hơn a. Bộ phân loại đầu tiên giúp xác định bit đầu tiên của nhãn, bộ thứ hai xác định bit tiếp theo,.... Cách làm này sử dụng một số lượng nhỏ nhất các bộ phân loại nhị phân. Tuy nhiên, một điểm dữ liệu chỉ được phân loại đúng khi mọi bộ phân loại nhị phân dự đoán đúng bit tương ứng. Hơn nữa, nếu số nhãn không phải là lũy thừa của hai, mã nhị phân nhận được có thể không tương ứng với nhãn nào.



Hình 14.8. Ví dụ về phân phối của các tập dữ liệu trong bài toán phân loại đa lớp.

14.6.4. one-vs-rest

Kỹ thuật được sử dụng nhiều nhất là *one-vs-rest*[39]. Cụ thể, C bộ phân loại nhị phân được xây dựng tương ứng với các nhãn. Bộ thứ nhất xác định một điểm có nhãn thứ nhất hay không, hoặc xác suất để một điểm có nhãn đó. Tương tự, bộ thứ hai xác định điểm đó có nhãn thứ hai hay không hoặc xác xuất có nhãn thứ hai là bao nhiêu. Nhãn cuối cùng được xác định theo nhãn mà điểm đó rơi vào với xác suất cao nhất.

Hồi quy logistic trong thư viện scikit-learn có thể được áp dụng trực tiếp vào các bài toán phân loại đa lớp với kỹ thuật one-vs-rest. Với MNIST, ta có thể dùng hồi quy logistic kết hợp với one-vs-rest (mặc định) như sau:

```
X_train, X_test, y_train, y_test = \
train_test_split(X_all, y_all, test_size=10000)
model = LogisticRegression(C = 1e5) # C is inverse of lam
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy %.2f %%" % (100*accuracy_score(y_test, y_pred.tolist())))
```

Kết quả thu được tương đối thấp, khoảng 91.7%. Phương pháp KNN đơn giản hơn đã có độ chính xác khoảng 96%. Điều này chứng tỏ one-vs-rest không làm việc tốt trong trường hợp này.

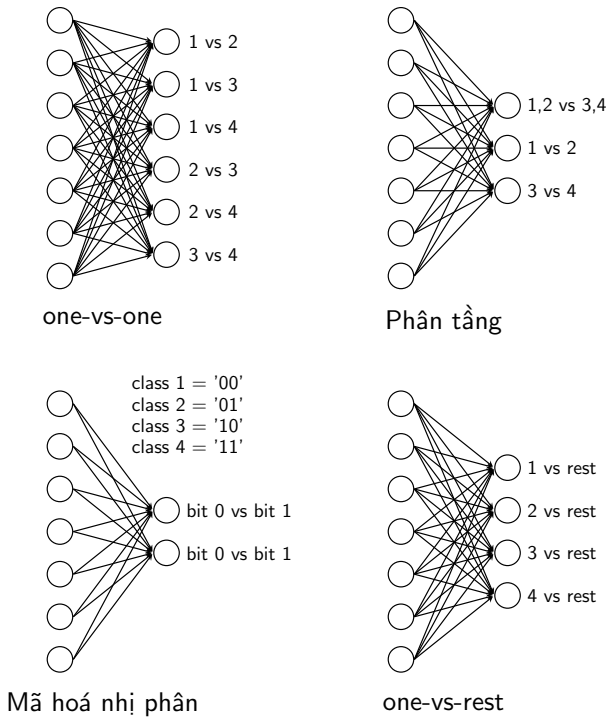
14.7. Thảo luận

14.7.1. Kết hợp các phương pháp trên

Trong nhiều trường hợp, ta cần kết hợp nhiều kỹ thuật trong số bốn kỹ thuật đã đề cập. Xét ba ví dụ trong Hình 14.8.

- Hình 14.8a: Cả bốn phương pháp trên đây đều có thể áp dụng được.

[39] Một số tài liệu gọi là *one-vs-all*, *one-against-rest*, hoặc *one-against-all*.



Hình 14.9. Mô hình neural network cho các kỹ thuật sử dụng các bộ phân loại nhị phân cho bài toán phân loại đa lớp.

- Hình 14.8b: One-vs-rest không phù hợp vì tập dữ liệu ở giữa và hợp của hai tập còn lại là không (gần) tách biệt tuyến tính. Lúc này, one-vs-one hoặc phân tầng phù hợp hơn.

- Hình 14.8c: Tương tự như trên, có ba tập dữ liệu thẳng hàng nên one-vs-rest sẽ không phù hợp. Trong khi đó, one-vs-one vẫn hiệu quả vì từng cặp nhãn dữ liệu là (gần) tách biệt tuyến tính. Tương tự, phân tầng cũng làm việc nếu ta phân chia các nhãn một cách hợp lý. Ta cũng có thể kết hợp nhiều phương pháp. Ví dụ, dùng one-vs-rest để tách nhãn ở hàng trên ra khỏi ba nhãn thẳng hàng ở dưới. Ba nhãn còn lại có thể tiếp tục được phân loại bằng các phương pháp khác. Tuy nhiên, khó khăn vẫn nằm ở việc phân nhóm như thế nào.

Với bài toán phân loại đa lớp, nhìn chung các kỹ thuật sử dụng các bộ phân loại nhị phân ít mang lại hiệu quả. Mời bạn đọc thêm Chương 15 và Chương 29 để tìm hiểu về các bộ phân loại đa lớp phổ biến nhất hiện nay.

14.7.2. Biểu diễn dưới dạng mạng neuron

Lấy ví dụ bài toán có bốn nhãn dữ liệu $\{1, 2, 3, 4\}$; ta có thể biểu diễn các kỹ thuật đã được đề cập dưới dạng mạng neuron như trong Hình 14.9. Mỗi nút ở tầng đầu ra thể hiện đầu ra của một bộ phân loại nhị phân.

Các mạng neuron này đều có nhiều nút ở tầng đầu ra, vector trọng số $\mathbf{w}$ đã trở thành *ma trận trọng số* $\mathbf{W}$. Mỗi cột của $\mathbf{W}$ tương ứng với vector trọng số của

một nút đầu ra. Các bộ phân loại nhị phân này có thể được xây dựng đồng thời. Nếu chúng là các bộ hồi quy logistic, công thức cập nhật theo SGD:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta(a_i - y_i)\mathbf{x}_i \tag{14.20}$$

có thể được tổng quát thành

$$\mathbf{W} \leftarrow \mathbf{W} - \eta\mathbf{x}_i(\mathbf{a}_i - \mathbf{y}_i)^T. \tag{14.21}$$

Với $\mathbf{W}, \mathbf{y}_i, \mathbf{a}_i$ lần lượt là ma trận trọng số, vector đầu ra thực sự và vector đầu ra dự đoán ứng với dữ liệu $\mathbf{x}_i$. Chú ý rằng vector $\mathbf{y}_i$ là một vector nhị phân, vector $\mathbf{a}_i$ gồm các phần tử nằm trong khoảng $(0, 1)$.

Chú ý: Số hạng thứ hai trong (14.21) không thể là $(\mathbf{a}_i - \mathbf{y}_i)\mathbf{x}_i^T$ vì ma trận này khác chiều với $\mathbf{W}$. Số hạng này cần là tích của hai vector: vector thứ nhất cần có cùng số hàng với $\mathbf{W}$, tức chiều của dữ liệu $\mathbf{x}_i$; vector thứ hai cần phù hợp với số cột của $\mathbf{W}$, tức số nút ở tầng đầu ra.